

Decisões de Arquitetura

Desenho da solução proposta antes da implementação — componentes, fluxo e cada decisão técnica no formato contexto → decisão → alternativa descartada → consequência.

ÍNDICE

PDF Requisitos do desafio	ADR Rate limiting
00 Resumo executivo	ADR Retry sem abstração
01 Fluxo ponta a ponta	ADR Sem defaults de credencial
ADR Camadas / Ports & Adapters	ADR Correlation ID
ADR Idempotência do consumer	02 Perguntas frequentes
ADR Dual-write & Outbox	03 Riscos e evolução futura
ADR Cache-aside	

Requisitos do desafio a atender

Pontos que o enunciado do desafio pede pra endereçar na etapa de System Design, com a resposta da proposta pra cada um.

“Quais componentes fariam parte da solução.”

→ FastAPI na entrada HTTP, MySQL + SQLAlchemy na persistência, Kafka na mensageria assíncrona, um consumer dedicado no processamento e Redis no cache de leitura e no rate limit.

“Como seria o fluxo da informação entre eles.”

→ Detalhado na seção "Fluxo ponta a ponta" logo abaixo.

“Onde utilizaria Kafka, MySQL e Redis.”

→ Kafka desacopla a criação do processamento; MySQL é a fonte da verdade do estado da solicitação; Redis cobre dois papéis independentes — cache de leitura e limite de taxa.

“Como trataria falhas durante o processamento.”

→ Retry limitado com backoff, fila de dead-letter após esgotar as tentativas, e status **FAILED** visível — ver ADR-006.

“Como evitaria que uma mesma mensagem fosse processada mais de uma vez.”

→ Aceitando entrega at-least-once e tornando a transição de status idempotente no domínio — ver ADR-002.

“Como organizaria as responsabilidades entre domínio, aplicação e infraestrutura.”

→ Arquitetura Hexagonal (Ports & Adapters), com a dependência sempre apontando pra dentro — ver ADR-001.

“Não existe uma estratégia obrigatória para expiração ou invalidação do cache.”

→ Cache-aside na leitura, invalidação por remoção da chave na escrita — ver ADR-004.

RESUMO EXECUTIVO

Proponho um sistema de processamento assíncrono de solicitações: a API persiste em MySQL e publica um evento no Kafka; um consumer separado processa e atualiza o status; a consulta usa Redis em cache-aside. Vou separar domínio e infraestrutura em camadas — Ports & Adapters — pra testar a regra de negócio sem depender de Kafka, MySQL ou Redis reais. A partir daí, cada decisão técnica abaixo é um trade-off deliberado, não a única forma de resolver.

Fluxo ponta a ponta

`POST /requests` → valida payload → grava **PENDING** no MySQL → publica `requests.created` no Kafka → 201

Consumer consome a mensagem → aplica a regra (≤ 1000 → **APPROVED**, > 1000 → **MANUAL_REVIEW**) → grava novo status → invalida a chave no Redis

`GET /requests/{id}` → Redis hit → resposta • Redis miss → MySQL → popula Redis → resposta

Falha esgotada → mensagem original vai pro tópico **DLQ** → status vira **FAILED**

ADR-001 Camadas em Ports & Adapters (Hexagonal)

Definida

CONTEXTO

A regra de negócio (faixa de valor decide o status) precisa ser testável sem depender de banco, fila ou cache real, e a tecnologia de infraestrutura pode mudar sem exigir

reescrever a regra.

DECISÃO

Domínio sem nenhuma dependência externa (dataclass + enum puro); aplicação com use cases que conhecem só *Protocols* (portas) — nunca SQLAlchemy, Kafka ou Redis diretamente; infraestrutura implementa essas portas e hospeda os adapters de entrada (router HTTP, consumer).

ALTERNATIVA DESCARTADA

MVC direto, com o use case chamando SQLAlchemy e o Kafka producer diretamente — mais rápido de escrever, mas acopla a regra de negócio ao framework e ao driver de banco, e testar exige subir infraestrutura real.

CONSEQUÊNCIA

A expectativa é rodar a suíte de testes em segundos, com SQLite em memória e dublês de Kafka/Redis, sem depender de Docker. Trocar MySQL por outro banco, ou Kafka por outro broker, deve tocar só a camada de infraestrutura.

JUSTIFICATIVA

Separo em domínio, aplicação e infraestrutura porque quero testar a regra de negócio sem precisar subir Kafka, MySQL ou Redis reais — só assim a suíte roda rápido e sem infraestrutura externa.

ADR-002 **At-least-once + idempotência, não exactly-once**

Definida

CONTEXTO

Kafka nativamente garante entrega, não garante processamento único — a mesma mensagem pode chegar duas vezes ao consumer.

DECISÃO

`enable_auto_commit=False`: o offset só é commitado depois que o status é persistido no MySQL. O use case de processamento verifica `status != PENDING` antes de agir — reprocessar uma solicitação já finalizada é no-op.

ALTERNATIVA DESCARTADA

Exactly-once delivery nativo do Kafka (transações de producer/consumer) — resolve na entrega, mas exige configuração de broker específica e adiciona latência desproporcional ao escopo do desafio.

CONSEQUÊNCIA

Reentrega da mesma mensagem (por queda do processo, rebalance, etc.) nunca duplica o efeito no banco.

JUSTIFICATIVA

Não busco exactly-once na entrega porque isso custa latência e exige broker específico. Prefiro at-least-once, mais barato, e torno a transição de status idempotente no domínio — resolve o mesmo problema com menos infraestrutura.

ADR-003 Dual-write MySQL ↔ Kafka: Outbox como evolução

Evolução planejada

CONTEXTO

Persistir no MySQL e publicar no Kafka são duas operações em sistemas distintos, sem transação distribuída cobrindo as duas.

DECISÃO

Aceitar a janela de risco no escopo do desafio (se o processo morrer entre o commit e o produce, a solicitação fica PENDING indefinidamente) e documentar o padrão **Transactional Outbox** como a solução correta: gravar o evento numa tabela outbox na mesma transação do domínio, um relay assíncrono publica depois.

ALTERNATIVA DESCARTADA

Implementar o Outbox agora — tecnicamente correto, mas adiciona uma tabela, uma migration e um processo relay novo; complexidade desproporcional ao escopo e ao prazo do teste.

CONSEQUÊNCIA

Risco real e conhecido, de baixa probabilidade (banco e broker no mesmo host, raramente caem no meio de um produce) — decisão consciente de escopo, não descuido.

JUSTIFICATIVA

Sei que existe uma janela entre o commit no banco e o publish no Kafka. A solução correta em produção é Outbox transacional — registro como próximo passo em vez de implementar agora, porque o ganho não compensa a complexidade extra nesse escopo.

ADR-004 Cache-aside com invalidação por DELETE

Definida

CONTEXTO

O GET precisa refletir o status mais recente assim que o consumer processa a solicitação, sem consultar o MySQL a cada leitura.

DECISÃO

Leitura em cache-aside com TTL de 15 min. Na escrita, o consumer **deleta** a chave em vez de atualizá-la.

ALTERNATIVA DESCARTADA

Write-through (consumer escreve o novo valor direto no Redis) — se essa escrita falhar no meio do caminho, Redis e MySQL ficam divergentes sem nenhum mecanismo de correção automática.

CONSEQUÊNCIA

Se o DELETE falhar, o pior caso é o TTL expirar sozinho em até 15 min — o dado nunca fica errado, só potencialmente desatualizado por uma janela curta e limitada.

JUSTIFICATIVA

Escolho invalidar em vez de atualizar porque, se o DELETE falhar, o TTL resolve sozinho. Write-through com falha parcial não tem essa autocorreção — pode ficar errado indefinidamente.

ADR-005 **Rate limiting — não pedido, adicionado por custo baixo**

Definida

CONTEXT0

POST /requests é o único endpoint que cria estado persistente — sem limite, um loop de chamadas enche o banco.

DECISÃO

INCR + EXPIRE num pipeline Redis transacional (MULTI/EXEC), contador por IP de origem.

ALTERNATIVA DESCARTADA

INCR e EXPIRE como dois comandos separados — sob concorrência, existe uma janela em que o EXPIRE nunca é aplicado e o contador vira "imortal".

CONSEQUÊNCIA

Janela deslizante (o TTL renova a cada requisição), não fixa — suficiente contra abuso, mais simples que um script Lua atômico para janela fixa estrita.

JUSTIFICATIVA

Não é requisito, mas o Redis já está no stack disponível e o custo é baixo — um endpoint que persiste estado sem limite nenhum é convite a abuso.

ADR-006 **Retry linear no consumer, sem abstração**

Definida

CONTEXT0

Falhas transitórias no processamento (banco momentaneamente fora, por exemplo) não deveriam derrubar a mensagem direto pra DLQ.

DECISÃO

3 tentativas, backoff linear com teto de 30s ($\min(\text{tentativa}, 30)$) — bem abaixo do `max.poll.interval.ms` padrão do Kafka (300s), evitando que o consumer seja removido do grupo por inatividade.

ALTERNATIVA DESCARTADA

Uma classe `RetryStrategy` genérica ou decorador configurável — over-engineering para uma única implementação concreta com parâmetros fixos (YAGNI).

CONSEQUÊNCIA

Se o backoff precisar mudar (exponencial, jitter), é uma mudança de uma linha, não uma migração de abstração.

JUSTIFICATIVA

Não crio abstração de retry porque só existe uma implementação concreta. Se precisar virar exponencial com jitter no futuro, é trocar uma linha — não vale abstrair agora.

ADR-007 Configuração sem defaults de credencial

Definida

CONTEXTO

Um valor default para `DATABASE_URL` é conveniente em dev, mas é o primeiro grep numa revisão de segurança.

DECISÃO

`database_url` obrigatório no `Settings` (pydantic-settings) — a aplicação recusa subir com `ValidationError` claro se a variável não estiver definida.

ALTERNATIVA DESCARTADA

Default apontando pro MySQL local do Docker Compose — funciona em dev, mas normaliza o hábito de ter credencial hardcoded no código.

CONSEQUÊNCIA

Falha rápida e explícita na inicialização em vez de silenciosamente usar uma credencial errada.

JUSTIFICATIVA

Prefiro que a aplicação recuse subir sem `DATABASE_URL` a ter um default que alguém esquece de trocar em produção.

CONTEXTO

Uma solicitação atravessa dois processos (API e consumer) via Kafka — sem um ID comum, os logs de cada lado não se conectam.

DECISÃO

X-Correlation-Id gerado (ou recebido) na borda HTTP via middleware, propagado por ContextVar, embutido no payload Kafka e lido pelo consumer. Todos os logs saem em JSON estruturado.

ALTERNATIVA DESCARTADA

Tracing distribuído completo (OpenTelemetry) — citado como evolução natural, não implementado por não ser requisito e aumentar a superfície de infraestrutura do desafio.

CONSEQUÊNCIA

Um grep pelo ID deve reconstruir a jornada inteira: request na API e processamento no consumer, dois processos, uma história só.

JUSTIFICATIVA

Com log em JSON e correlation_id, um grep por um ID só mostra a jornada inteira — request na API e processamento no consumer — mesmo sendo dois processos separados.

Perguntas frequentes

At-least-once vs. exactly-once vs. at-most-once?

At-least-once: pode reentregar, nunca perde. At-most-once: pode perder, nunca duplica. Exactly-once: nem perde nem duplica — caro, exige suporte do broker. Uso at-least-once + idempotência, que dá o efeito de exactly-once sem o custo.

O que é Transactional Outbox, na prática?

Grava o evento numa tabela outbox dentro da mesma transação SQL que grava o estado do domínio. Um processo relay separado lê essa tabela e publica no Kafka. A atomicidade vem da transação MySQL, não de coordenação entre dois sistemas.

Se o Kafka cair no meio de um POST, o que acontece?

O producer tenta 3 vezes e lança exceção; o handler global responde 500. Mas a solicitação já foi persistida como PENDING antes da publicação — o dado não se perde, fica pendente até ser reprocessado. É a mesma janela do ADR-003.

Por que Protocol e não uma classe abstrata (ABC) nas portas?

Protocol é tipagem estrutural: um fake não precisa herdar de nada, só ter os métodos certos. Menos acoplamento entre o código de teste e o de produção.

Por que dataclass e não Pydantic no domínio?

O domínio não deveria depender de nenhum framework — nem Pydantic. Dataclass é da stdlib; reforça que a regra de negócio não conhece a camada web.

Como isso escalaria horizontalmente?

Múltiplas instâncias do consumer no mesmo group_id dividem as partições do tópico automaticamente (rebalanceamento nativo do Kafka). A API é stateless, escala atrás de um load balancer.

Riscos conhecidos e evolução futura

- **Autenticação e autorização** ficam fora do escopo inicial — o campo `customer_id` chega livre no payload. Em produção, viria de um token (JWT) validado na borda, não do corpo da requisição.
- **Rate limit por IP**, não por conta — suficiente contra abuso simples, mas insuficiente contra múltiplos usuários atrás do mesmo NAT ou um atacante com IPs rotativos. Evolução natural é limitar por `customer_id` extraído do token, com IP como limite secundário.
- **Transactional Outbox** não entra na primeira versão — ver ADR-003. Fica documentado como o próximo passo assim que a janela de dual-write se tornar um risco relevante em produção.

Desenho da solução — Parte 1 do desafio técnico, complementado pelas decisões de implementação da Parte 2.